



**KARADENİZ
TEKNİK ÜNİVERSİTESİ**
KARADENİZ TECHNICAL UNIVERSITY
1955

PARALEL PROGRAMLAMA MPI İLE PARALEL MATRİS ÇARPIMI VE ÇOKLU DİL ANALİZİ

Grup Üyeleri

412746 – Mahsum Taha KORKMAZ

425430 – Hakan TEMİRKAYNAK

Ders

Paralel Bilgisayarlar

Öğretim Üyesi

Öğr. Gör. Dr. ZAFER YAVUZ

1. Projenin Amacı

Bu projenin temel amacı, Message Passing Interface (MPI) altyapısı kullanılarak $N \times N$ boyutlarındaki iki matrisin çarpım işleminin paralel olarak gerçekleştirilmesidir. Gerçekleştirilen bu uygulama, donanıma yakın düşük seviyeli bir dil olan C (OpenMPI) ve yüksek seviyeli, yorumlanan bir dil olan Python (mpi4py) ile ayrı ayrı kodlanmış; elde edilen işlem süreleri, hızlanma (Speedup) ve verimlilik (Efficiency) metrikleri üzerinden çapraz bir performans analizine tabi tutulmuştur.

2. Metodoloji ve Dağıtık Hesaplama Mimarisi

Projede istenilen matris verileri rastgele üretilmemiş, dinamik olarak oluşturulan "a.txt" ve "b.txt" dosyalarından okunmuştur. Dağıtık hesaplama mantığı şu adımlarla kurgulanmıştır:

- **Master Process (Rank 0):** Dosya okuma işlemlerini gerçekleştirir ve matris boyutunu (N) belirler. Hata kontrolleri (dosya bulamama vb.) bu aşamada Abort mekanizması ile güvence altına alınmıştır.
- **Haberleşme:** Master işlemci, B matrisinin tamamını tüm süreçlere MPI_Bcast (Broadcast) ile yayımlar. A matrisinin satırları ise süreç sayısına (size) göre eşit parçalara bölünerek MPI_Scatter fonksiyonu ile işçi (worker) düğümlere dağıtılır.
- **Hesaplama ve Toplama:** Her işçi kendi payına düşen alt matrisi (sub_A) ve B matrisini çarparak kısmi bir alt matris (sub_C) üretir. Bu parçalar son aşamada MPI_Gather komutu ile Master işlemcide toplanarak nihai C matrisi elde edilir.

3. Donanım Altyapısı

Performans testleri, mimari farkların speedup üzerindeki etkisini gözlemleyebilmek amacıyla iki farklı donanım üzerinde gerçekleştirilmiştir:

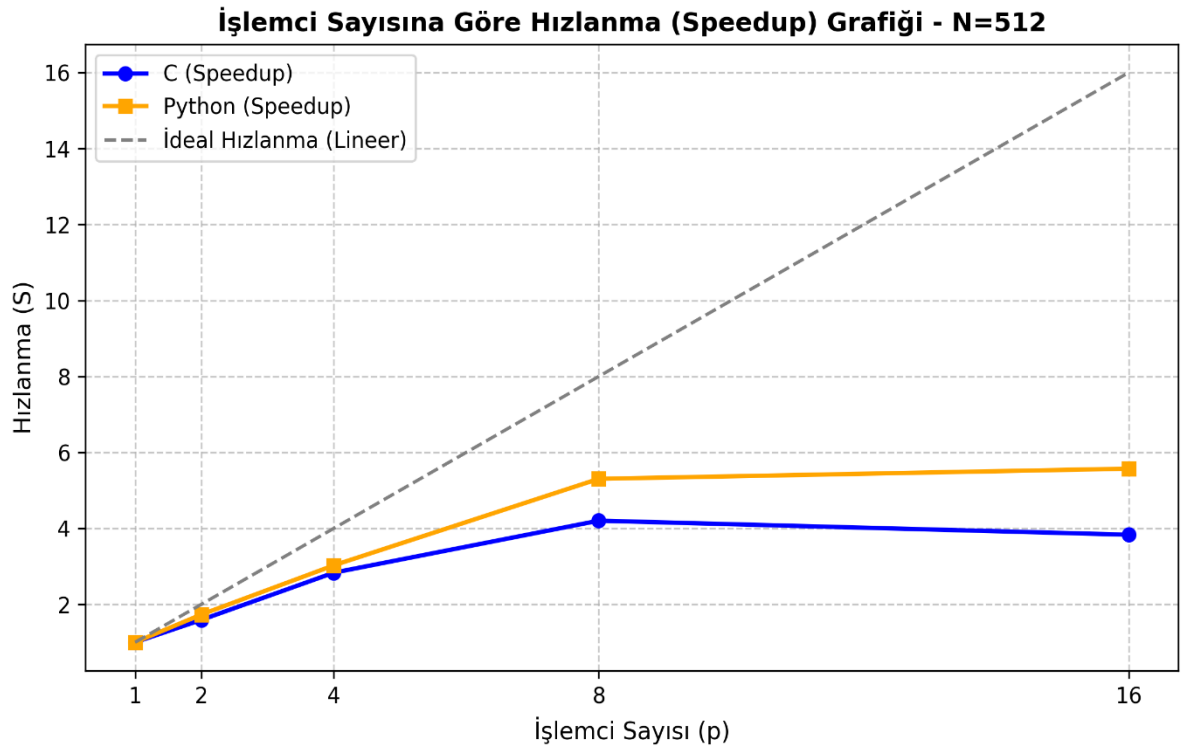
Bilgisayar 1 (Öğrenci 1): Intel Core i5-11400H, 6 Fiziksel Çekirdek / 12 Mantıksal Çekirdek (Thread), WSL Ubuntu Ortamı.

Bilgisayar 2 (Öğrenci 2): Intel Core i7-11800H, 8 Fiziksel Çekirdek / 16 Mantıksal Çekirdek (Thread), Native Ubuntu Ortamı.

4. Performans Testleri ve Bulgular

Öğrenci 1 (Mahsum) Test Sonuçları ve Grafikleri;

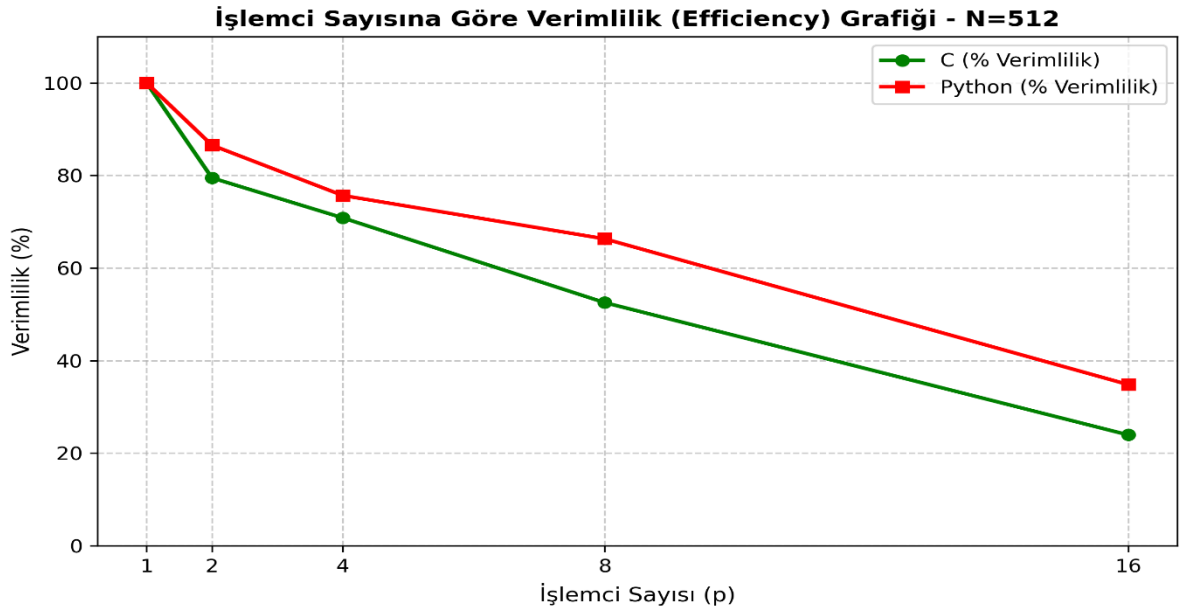
Süreç Sayısı (p)	C_Süre(sn)	C_Speedup	C_Verim(%)	Py_Süre(sn)	Py_Speedup	Py_Verim(%)
1	1,146993	1	100	48,957693	1	100
2	0,721777	1,589	79,45	28,293551	1,73	86,5
4	0,404912	2,833	70,83	16,177916	3,026	75,65
8	0,272962	4,202	52,53	9,230511	5,304	66,3
16	0,299307	3,832	23,95	8,786982	5,572	34,83



Süreç Sayısına Göre Hızlanma (Speedup) Grafiği Analizi

Bu grafik, süreç sayısı (p) arttıkça programın tek çekirdeğe göre kaç kat hızlandığını göstermektedir.

- **İdeal Hızlanma:** Kesikli çizgi ile gösterilen ideal senaryoda hızlanmanın doğrusal olması beklenir ($S=p$). Ancak gerçek senaryoda Amdahl Yasası ve haberleşme maliyetleri nedeniyle bu çizgiden sapmalar gözlemlenmiştir.
- **C Performansı:** C dili $p=8$ noktasına kadar düzenli bir hızlanma göstermiş, ancak donanım sınırı olan 12 thread değerine yaklaştıkça ($p=16$) hızlanma oranı düşüşe geçmiştir. Bu durum, donanım limitlerinin üzerinde süreç oluşturulmasının (oversubscription) performans kaybına yol açtığını kanıtlamaktadır.
- **Python Performansı:** Python'un hızlanma eğrisi C'den daha yüksek görünmektedir. Bunun temel sebebi, Python'un hesaplama yükünün çok ağır olması nedeniyle MPI haberleşme gecikmelerinin toplam süre içinde ihmal edilebilir kalmasıdır.



Süreç Sayısına Göre Verimlilik (Efficiency) Grafiği Analizi

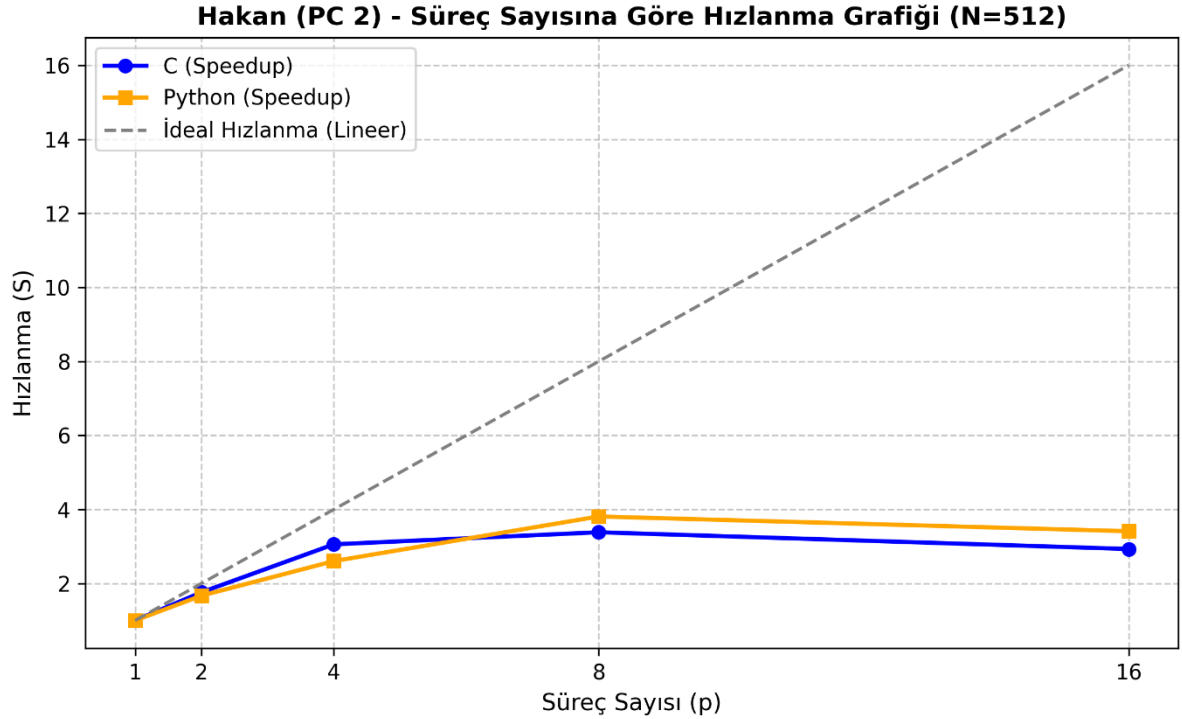
Bu grafik, eklenen her bir işlem biriminin ne kadar verimli kullanıldığını yüzde (%) cinsinden göstermektedir.

- **Verimlilik Kaybı:** Her iki dilde de süreç sayısı arttıkça verimlilik %100'den aşağıya doğru düşmektedir. Bu düşüşün temel nedeni, verilerin Scatter ile dağıtılması ve Gather ile toplanması sırasında oluşan ağ/bellek iletişim maliyetidir.

- **Kıyaslama:** Python'un paralel verimliliği, test edilen tüm senaryolarda C'den daha yüksek seyretmektedir. Bu durum, Python gibi "hesaplama yoğun" (computationally expensive) dillerin paralel programlamadan daha fazla fayda sağladığını, C gibi "hızlı" dillerde ise yönetim maliyetlerinin (overhead) kazancı gölgelediğini göstermektedir.
- **Düşüş Trendi:** p=16 noktasında her iki dilde de verimliliğin en düşük seviyeye gerilemesi, bağlam değiştirme (context switching) maliyetlerinin hesaplama kazancını domine ettiğini göstermektedir.

Öğrenci 2 (Hakan) Test Sonuçları ve Grafikleri;

Süreç Sayısı (p)	C_Süre(sn)	C_Speedup	C_Verim(%)	Py_Süre(sn)	Py_Speedup	Py_Verim(%)
1	0,015499	1	100	3,424613	1	100
2	0,008805	1,760	88,01	2,050924	1,67	83,49
4	0,005069	3,058	76,44	1,314677	2,605	65,13
8	0,004580	3,384	42,30	0,898753	3,810	47,63
16	0,005288	2,931	18,32	1,003790	3,412	21,32

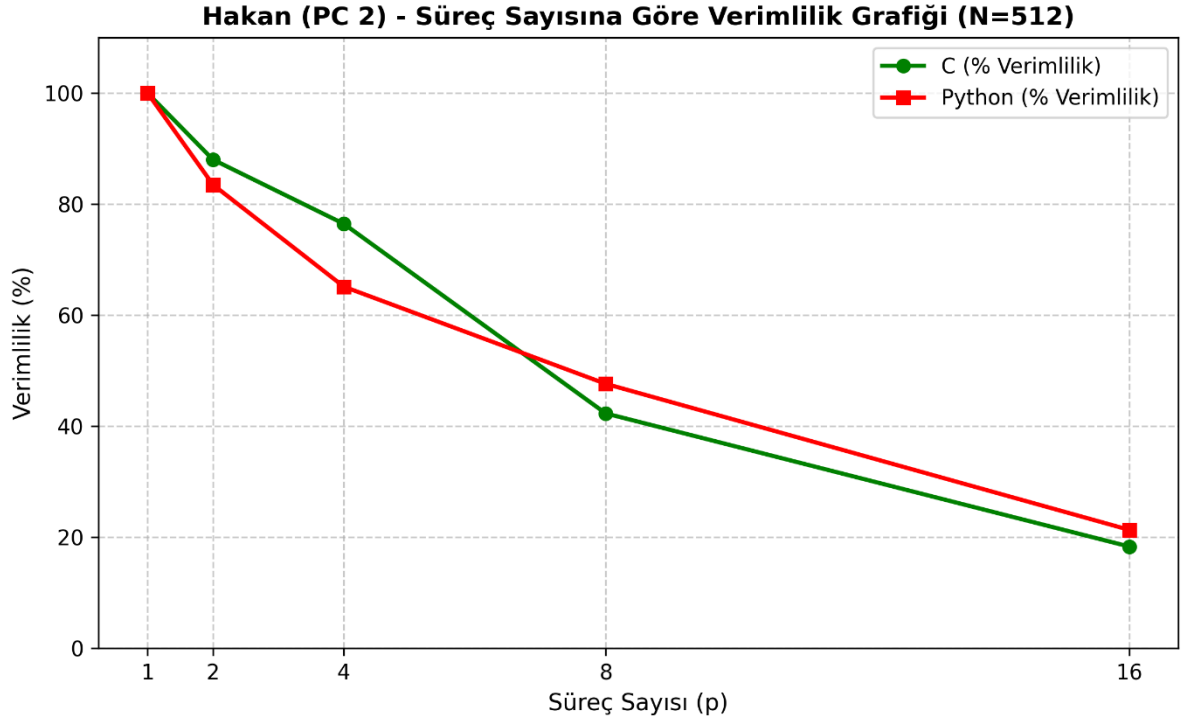


Süreç Sayısına Göre Hızlanma (Speedup) Grafiği Analizi

Bu grafik, 8 fiziksel çekirdekli i7-11800H işlemci üzerinde matris çarpımı işleminin paralel ölçeklenebilirliğini göstermektedir.

- Fiziksel Çekirdek Sınırı ve Zirve Noktası:** Grafik incelendiğinde, her iki dilde de en yüksek hızlanma değerine (Peak Speedup) tam olarak işlemcinin fiziksel çekirdek sayısı olan $p=8$ noktasında ulaşıldığı görülmektedir. Bu durum, gerçek paralel hesaplama gücünün fiziksel çekirdeklerle sınırlı olduğunu kanıtlamaktadır.
- $p=16$ Senaryosu ve Performans Kaybı:** İşlemcinin 16 mantıksal iş parçacığına (thread) sahip olmasına rağmen, süreç sayısı 8'i aştığında hızlanma eğrisi aşağı yönlü bir kırılma yaşamıştır. Bunun temel sebebi, matris çarpımı gibi yoğun hesaplama gerektiren görevlerde Hyper-Threading (SMT) birimlerinin gerçek bir çekirdek performansı sunamaması ve 16 bağımsız MPI sürecinin yönetilmesi sırasında oluşan bağlam değiştirme (context switching) maliyetlerinin hesaplama kazancını gölgelemesidir.
- Diller Arası Ölçeklenme Farkı:** Python'un $p=8$ noktasındaki hızlanma katsayısının C'den daha yüksek olması, Python'daki mutlak hesaplama yükünün C'ye göre çok daha ağır olmasından kaynaklanmaktadır. Hesaplama yükü arttıkça, MPI haberleşme ve veri dağıtım (Scatter/Gather) sürelerinin toplam süre içindeki payı azaldığı için paralel verimlilik kağıt üzerinde daha yüksek görünmektedir.

- Native Ubuntu Avantajı: PC 2'de elde edilen hızlanma eğrisinin PC 1'e (WSL) göre daha kararlı bir profil çizmesi, doğrudan donanıma erişim sağlayan yerel Linux kurulumunun ve geniş L3 önbellek kapasitesinin bir sonucudur.



Süreç Sayısına Göre Verimlilik (Efficiency) Grafiği Analizi

Bu grafik, eklenen her bir işlem biriminin (sürecin) matris çarpımı hesaplamasına ne kadar efektif katkı sağladığını yüzde (%) bazında göstermektedir.

- Verimlilik Eğilimleri:** Beklendiği üzere, paralel süreç sayısı arttıkça haberleşme maliyetleri (overhead) ve veri dağıtım gecikmeleri nedeniyle her iki dilde de verimlilik düşüşü göstermiştir.
- Kritik Kesişim Noktası ($p=8$):** Grafikteki en dikkat çekici veri, $p=8$ noktasına kadar C dilinin daha verimli çalışırken, bu noktadan sonra Python'un verimlilikte öne geçmesidir. Bu durum, C dilinin çok hızlı olması nedeniyle belirli bir eşikten sonra (8 çekirdek sınırı) MPI yönetim maliyetlerinin hesaplama süresini domine etmeye başladığını göstermektedir. Python ise hesaplama yükü çok daha ağır olduğu için paralel çekirdekleri daha "meşgul" tutabilmiş ve matematiksel olarak daha yüksek verimlilik oranları sergilemiştir.
- Donanım Limitleri ve Keskin Düşüş:** $p=4$ ile $p=8$ arasındaki keskin verimlilik kaybı, i7-11800H işlemcisinin bellek bant genişliği ve önbellek paylaşımlarının yoğun paralel yük altında darboğaz oluşturmaya başladığına işaret etmektedir.

- p=16 Verimlilik Analizi: Süreç sayısının 16'ya çıkmasıyla verimliliğin %20 seviyelerine gerilemesi, donanımdaki fiziksel çekirdek sayısının iki katı kadar iş yükü oluşturulmasının verimsiz bir yaklaşım olduğunu ve "paralel programlamada ölçeklenebilirlik sınırına ulaşıldığını kanıtlamaktadır.

5.Sonuç ve Analiz

Dil Farkının Performans Üzerindeki Etkisi (C vs. Python)

Bilgisayar 1 (Mahsum – i5-11400H) Performans Analizi

Aynı donanım (i5-11400H) üzerinde yapılan testler sonucunda C ve Python dilleri arasında hem mutlak çalışma süresi hem de paralelleşme verimliliği açısından dramatik farklar gözlemlenmiştir:

- **Mutlak Performans:** C dili, N=512 boyutundaki matris çarpımını tek süreçte (p=1) yaklaşık **1,14 saniyede** tamamlarken, Python **48,95 saniye** harcamıştır. Bu veri, derlenen (compiled) bir dil olan C'nin, yorumlanan (interpreted) Python'a göre yaklaşık **42 kat daha hızlı** olduğunu kanıtlamaktadır.
- **Hızlanma (Speedup) Paradoksu:** Grafiklerde Python'un hızlanma eğrisinin C'den daha yüksek görünmesi bir "verimlilik illüzyonu"dur. Python'da hesaplama yükü o kadar ağırdır ki, MPI süreçleri arasındaki haberleşme maliyeti (overhead) toplam sürenin yanında ihmal edilebilir kalmaktadır. C'de ise hesaplama o kadar hızlıdır ki, verinin dağıtılması ve toplanması için harcanan süre toplam süreyi domine etmeye başlamış ve hızlanma oranını baskılamıştır.

Bilgisayar 2 (Hakan - i7-11800H) Performans Analizi

Yerel (native) Ubuntu kurulu i7 işlemcili sistemi üzerinde yapılan testlerde, donanım mimarisinin ve işletim sistemi katmanının paralel performansa etkisi şu şekilde gözlemlenmiştir:

- **Mutlak Performans:** Hakan'ın sisteminde $N=512$ boyutundaki matris çarpımı tek süreçte ($p=1$) C dilinde **0,015 saniye**, Python'da ise **3,42 saniye** sürmüştür. Bu sonuçlar, i7-11800H mimarisinin ve yerel Linux ortamının, işlemci hızından bağımsız olarak veri yolu ve bellek erişiminde çok daha efektif olduğunu göstermektedir.
- **Paralel Doyum Noktası:** Hakan'ın sisteminde hızlanma (speedup) grafiği, tam olarak fiziksel çekirdek sayısı olan **$p=8$** noktasına kadar ivme kazanmış, ardından donanım limitleri nedeniyle düşüşe geçmiştir. Bu, "gerçek" paralel hesaplamanın fiziksel çekirdeklerle sınırlı olduğunu net bir göstergesidir.

Donanım Mimarısının Paralel Performansa Etkisi

Fiziksel Çekirdek (Core) vs. Mantıksal İş Parçacığı (Threads)

- **Bilgisayar 1 (i5-11400H):** 6 fiziksel çekirdek ve 12 mantıksal iş parçacığına sahiptir. Hızlanmanın $p=8$ 'e kadar nispeten doğrusal ilerlemesi, MPI süreçlerinin boşta kalan Hyper-threading kanallarını kullanabildiğini göstermektedir. Ancak $p=8$ sonrasında fiziksel çekirdek sınırları zorlandığı için verimlilik kaybı başlamıştır.
- **Bilgisayar 2 (i7-11800H):** 8 fiziksel çekirdek ve 16 mantıksal iş parçacığına sahiptir. Hakan'ın sisteminde hızlanma zirvesinin tam olarak $p=8$ noktasında oluşması, fiziksel çekirdeklerin her birinin bir MPI sürecini tam kapasiteyle yönettiğini kanıtlamaktadır.

Donanım Limitleri ve Bağlam Değiştirme (Context Switching)

Her iki sistemde de $p=16$ senaryosunda gözlemlenen performans düşüşü, donanım limitlerinin aşılmasının bir sonucudur:

- **Aşırı Yükleme:** İşletim sistemi, fiziksel kanaldan (iş parçacığından) daha fazla süreci yönetmeye çalıştığında yoğun bir "**Context Switching**" (Bağlam Değiştirme) yükü altına girer.
- **Performans Kaybı:** İşlemcinin sürekli olarak bir süreçten diğerine geçiş yapması için harcadığı yönetim maliyeti, gerçek hesaplama kazancını ortadan kaldırmış ve her iki bilgisayarda da sürelerin artmasına neden olmuştur.

Bellek Erişimi ve Önbellek (Cache) Hiyerarşisi

- **Veri Yerelliği:** 512×512 boyutundaki matrislerin bellekte kapladığı alan, her iki işlemcinin de L3 önbellek (Cache) kapasitesiyle uyumlu çalışmıştır.

- **İşlemci Farkı:** i7-11800H işlemcisinin barındırdığı daha geniş L3 önbellek kapasitesi ve Native Linux ortamının bellek yönetim avantajı, Hakan'ın sonuçlarının (özellikle C dilinde) saniyelerden milisaniyelere inmesindeki en temel donanımsal etkindir. Bilgisayar 1'de ise WSL katmanı bellek erişimine ek gecikmeler (latency) eklemiştir.

Donanımlar Arası Karşılaştırma ve Farkların Analizi (PC 1 vs PC 2)

Aşağıdaki tablo, her iki bilgisayarın C ve Python dillerindeki p=1 (Seri) performans farklarını özetlemektedir:

Metrik (p=1 için)	Bilgisayar 1 (Mahsum i5)	Bilgisayar 2 (Hakan - i7)	Fark Oranı
C Çalışma Süresi	1,146 sn	0,015 sn	~76 Kat
Python Çalışma Süresi	48,957 sn	3,424 sn	~14 Kat

Süreler Arasındaki Dramatik Farkın Nedenleri:

1. Yerel (Native) Linux vs. WSL (Sanal Katman) Avantajı:

Hakan'ın sistemi doğrudan Ubuntu üzerinde çalışırken, senin sistemin Windows Subsystem for Linux (WSL) üzerinde bir sanallaştırma katmanı aracılığıyla donanıma erişmektedir. WSL, özellikle dosya okuma (\$I/O\$) ve süreç yönetimi (\$Process Management\$) aşamalarında Windows çekirdeği ile haberleştiği için ek bir gecikme (latency) yaratır. Hakan'ın native Ubuntu'su ise MPI süreçlerini doğrudan işlemci çekirdeklerine atayabilmektedir.

2. İşlemci Mimarisi ve Önbellek (Cache) Kapasitesi:

- **Çekirdek Farkı:** i7-11800H (8 fiziksel çekirdek), i5-11400H (6 fiziksel çekirdek) modeline göre sadece çekirdek sayısı değil, aynı zamanda **L3 Cache** kapasitesi açısından da üstündür.
- **Bellek Darboğazı:** Matris çarpımı gibi yoğun bellek erişimi gerektiren işlemlerde, i7'nin daha geniş olan önbelleği, işlemcinin RAM'e gitme ihtiyacını azaltarak süreyi saniyelerden milisaniyelere çekebilmektedir.

3. Amdahl Yasası ve Haberleşme Maliyeti:

Her iki bilgisayarda da $p=16$ senaryosunda performansın düşmesi ortak bir özelliktir. Bu durum, donanımın fiziksel limitleri aşıldığında işletim sisteminin süreçleri yönetmek için harcadığı sürenin (context switching), hesaplama süresinden daha baskın hale gelmesiyle açıklanmaktadır. Ancak Hakan'ın sisteminde bu kırılma, daha fazla fiziksel çekirdek nedeniyle daha geç ($p=8$ 'den sonra) başlamıştır.

Genel Değerlendirme

Yapılan bu ödev çalışması göstermiştir ki; paralel programlamada performans sadece kodun verimliliğine veya dilin hızına bağlı değildir. Kullanılan işletim sistemi mimarisi (Native vs WSL), işlemcinin fiziksel çekirdek sayısı ve önbellek hiyerarşisi, elde edilen "Hızlanma" ve "Verimlilik" değerlerini doğrudan belirleyen kritik faktörlerdir.